

11.7.2026

Expt - 1

Aim:

To create a new process using the fork() system call and display the process ID (PID) and Parent Process ID (PPID).

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

void main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if (pid == 0)
```

```
        printf("I am a child process\n");
```

```
        printf("PID = %d\n", getpid());
```

```
        printf("Parent PID = %d\n", getppid());
```

```
    }
```

```
    else {
```

```
        printf("Parent Process\n");
```

```
        printf("PID = %d\n", getpid());
```

```
        printf("Parent PID = %d\n", getppid());
```

```
    }
```

```
    return 0;
```

```
}
```

Input:

Parent Process

PID = 1234

Parent PID = 1000

Child Process

PID = 12345

Parent ~~PID~~ ^{PID} = 1234

Result:

~~Thus~~ the child process was successfully deleted using the `fork()` system call.

11-7-2026

Experiment-2

Aim:

To copy the contents of one file into another using `system` system calls.

Code:

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{  
    int src, dst, n;
```

```
    char buff[100];
```

src = open ("Source.txt", O_RDONLY);

if (src < 0)

{ printf ("Source file not found \n");
return 0;

}

dst = open ("destination.txt", O_CREAT | O_
WRONLY | O_TRUNC, 0644);

while ((n = read (src, buffer, size of (buffer)))
> 0)

{ write (dst, buffer, n);

}

close (src);

close (dst);

printf ("File copied successfully \n");
return 0;

}

Output:

~~File copied successfully.~~

~~Result:~~

Thus the contents of one file were copied

Successfully using Linux system calls.

13.7.2026 Experiment - 3

Aim:

To implement First Come First Serve (FCFS) CPU Scheduling.

Code:

```
#include <stdio.h>
int main ()
{
    int bt[20], wt[20], tat[20];
    int i, n;
    float awt = 0, atat = 0;
    printf("Enter number of processes: ");
    scanf ("%d", &n);
    for (i=0; i<n; i++)
    {
        printf("Enter time of p%d: ", i+1);
        scanf ("%d", &bt[i]);
    }
}
```


wt[0] = 0;

for (i = 1; i <= n; i++)

wt[i] = wt[i-1] + bt[i];

for (i = 0; i < n; i++)

{
wt[i] = wt[i] + bt[i];

avg = wt[i];

at[i] = at[i] + wt[i];

}

printf ("| n | process | BT | WT | TAT | n");

for (i = 0; i < n; i++)

printf ("P%d \t T%d \t T%d \t T%d \t n",

i+1, bt[i], wt[i], at[i]);

printf ("| n | Average waiting time = 1/2 f",

avg/n);

printf ("| n | Average Turnaround time = 1/2

f", at[i]/n);

return 0;

}

Process BT WT TAT

P₁ 5 0 5

P₂ 3 5 8

P₃ 8 8 16

Avg waiting time = 4.33

Avg Turnaround time = 9.67

Result:

~~Thus~~ FCFS scheduling was implemented successfully.

13.7.2026 Experiment-4

Aim:

To implement Shortest Job First (SJF) Scheduling Algorithm.

Code:

```
#include <stdio.h>
```

```
int main ()
```

```
{  
    int bt[20], vt[20];
```

int ut[20], stat[20];

int i, j, n, temp;

float auct = 0, atot = 0;

printf ("Enter number of products: ");

scanf ("%d", &n);

for (i = 0; i < n; i++)

{
P[i] = i + 1;

printf ("Build time p.d: %d", i + 1);

scanf ("%d", &bt[i]);

}
for (i = 0; i < n; i++)

{
~~P[i] = i + 1;~~

~~printf ("Build time p.d: %d", i + 1);~~

~~scanf ("%d", &bt[i]);~~

}
for (i = 0; i < n; i++)

{
for (j = i + 1; j < n; j++)

{
if (bt[i] > bt[j])

{

at at (i);

printf ("Average waiting time = 1.2 s");
and (n);

printf ("Average Turnaround Time = 7.7 s");
at at (n);

return 0;

}

Result:

Process	BT	WT	TAT
P ₁	2	0	2
P ₂	3	2	5
P ₃	5	5	10
P ₄	8	10	18

Avg waiting time = 1.25

Avg Turnaround time = 8.75

~~Result:~~

Thus the Shortest Job First (SJF) scheduling algorithm was implemented successfully.

15.07.2026

Experiment - 5

Aim:
To implement Priority Scheduling
in C and create the process table.
highest priority first

Code

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, j;
```

```
int bt[20], ut[20], pt[20], p[20], temp;
```

```
float avg - ut = 0, avg - pt = 0;
```

```
printf("Enter the number of processes:");  
scanf("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
{ printf("Enter burst time and priority,
```

```
for process P-1-d: " i+1);
```

```
for (i=0; i<n; i++)
```

```
{ printf("Enter burst time and priority  
d process P-1-d: " i+1);
```

3 $for(i=0; i < n-1; i++)$

{ $for(j=i+1; j < n; j++)$

{ $if (P[i] > P[j])$

{ $temp = P[i];$

$P[i] = P[j];$

$P[j] = temp;$

$temp = b[i];$

$b[i] = b[j];$

$b[j] = temp;$

}

}

}

$w[0] = 0;$

$for(i=1; i < n; i++)$

$w[i] = w[i-1] + b[i-1];$

$for(i=0; i < n; i++)$

{ $stat[i] = w[i] + b[i];$

$avg_w += w[i];$

any other
which may be the subject of a
new or amended application
of law, etc.

1. Any person who is
in possession of a
document, etc.

2. Any person who is
in possession of a
document, etc.

3. Any person who is
in possession of a
document, etc.

4. Any person who is
in possession of a
document, etc.

5. Any person who is
in possession of a
document, etc.

6. Any person who is
in possession of a
document, etc.

7. Any person who is
in possession of a
document, etc.

8. Any person who is
in possession of a
document, etc.

9. Any person who is
in possession of a
document, etc.

10. Any person who is
in possession of a
document, etc.

Result:

~~As~~ the priority scheduling program was executed successfully and the waiting time and turnaround time for all processes will be obtained.

15.07.2026

Experiment-6

Aim:

To implement the preemptive priority scheduling algorithm in C and arrange the processes with the highest priority first.

Code:

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n, i, time = 0, completed = 0;
```

```
    int bt[10], st[10], at[10], pt[10];
```

```
    int wt[10], tat[10], ct[10];
```

```
    int highest priority;
```

```
    float avg wt = 0, avg tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
{
```

Minif ("Catal" and pholity, jul P.H.d.:", Plat [i], Plat [i], Plat [i]
&anf ("f.d.d.d.d", Plat [i], Plat [i]
of pl [i]; Plat [i] = Plat [i];

3
while (complexed < n)

{
highest = -1;
min pholity = aaaaa;

for (i = 0; i < n; i++)

{
if (at[i] < time of s at [i] > 0 of s pholity
min pholity)

{
min pholity = at [i];
highest = i;

}

3

if (highest == -1)

{
time ++;
continue;

avg wt (m);
 profit (m) exchange rate (m);
 1.2 ft m, avg wt (m);
 when 0;

Output:

Enter number of placettes: 3
 Enter arrival time, bulk time and
 priority for 1: 0 5 2

" " " " " "

" " " P2: 1 3 1

" " " " "

" " " P3: 2 1 3

AVG w.t = 3.00

AVG PAT = 7.00

~~Result:~~

Thus, the pre-emptive priority scheduling
 algorithm was implemented successfully in

17.07.2026

Experiment - 7

Aim:

to implement the non-preemptive shortest job first (SJF) scheduling algorithm in C and calculate the waiting time and turnaround time.

Code:

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
int bt [20], ut [20], tat [20], i, j, n, temp;
```

```
printf ("Enter the number of processes: ");
```

```
scanf ("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
{
```

```
printf ("Enter burst time for P-1: ", i+1);
```

```
scanf ("%d", &bt[i]);
```

```
}
```

```
for (i=0; i<n-1; i++)
```

```
{
```

```
for (j=i+1; j<n; j++)
```

```
{
```

if ($bt[i] > bt[j]$)

{
temp = $bt[i]$;
 $bt[i] = bt[j]$;
 $bt[j] = temp$;
}

}

}

$wt[0] = 0$

for ($i = 1; i \leq n; i++$)

$wt[i] = wt[i-1] + bt[i-1]$;

printf ("In Process 1st Bus at time 1 +
waiting time 1st bus at time 1 +");

for ($i = 2; i \leq n; i++$)

{

$wt[i] = wt[i-1] + bt[i-1]$;

printf ("In Process 1st Bus at time 1 +
waiting time 1st bus at time 1 +");

$i + 1, bt[i], wt[i], wt[i]$;

}

return 0;

}

Out put:

Enter the number of processes: 4

Enter burst time for P1: 6

" " " " P2: 8

" " " " P3: 7

~~Enter~~ " " " " P4: 3

Result:

~~Thus~~, the non-preemptive SJF Scheduling algorithm was implemented successfully and the waiting time and turnaround time.

17.07.2026 Experiment - 8

Aim:

To implement the Round Robin Scheduling algorithm in C and calculate the waiting time and turnaround time.

Code:

```
#include <stdio.h>
```

```
int main ()
```

```
{  
    int i, n, tq, remain, time = 0;
```

```
    int bt[10], st[10], ct[10], tat[10];
```


printf("Enter number of processes: ");
scanf("%d", &n);

remain = a;

for (i = 0; i < n; i++)

{

printf("Enter burst time for P.T.D: ");
scanf("%d", &bt[i]);

lt[i] = bt[i];

}

printf("Enter time quantum: ");

scanf("%d", &tg);

while (remain != 0)

{
for (i = 0; i < n; i++)

{
if (lt[i] > 0)

{
if (lt[i] > 0)

{
if (lt[i] <= tg)

{

time += lt[i];

lt[i] = 0;

```

    bt[i] = time;
    wt[i] = bt[i] - at[i];
    remain--;
}

```

```

}
//

```

```

{
    bt[i] = tq;
    time += 2 * tq;
}
}

```

```

}

```

```

}

```

```

{

```

```

printf("\n Process |> Burst time |> waiting time  
+ turnaround time |>");

```

```

for (i = 0; i < n; i++)

```

```

{

```

```

    printf("P-ID |> A.T |> B.T |> W.T |> T.T |>");
    i++;
    bt[i], wt[i], at[i], tat[i];
}

```

```

}

```

```

return 0;

```

```

}

```

Output:

Enter number of process: 3

Correct Bulb time for P1: 10
 " " " " P2: 5
 " " " " P3: 8
 7 Some Quations: 2

Result:
 Thus, the Round Robin Scheduling algorithm was implemented successfully in C.

20.7.2026

Experiment - 9

Aim:
 To implement interprocess communication (IPC) using shared memory in C.

Code:

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

```

```

int main()
{

```

```


```

```

    key_t key = 1234;

```

```

    int shared;

```

```

    char *data;

```

... ..
... ..
... ..
... ..
... ..
... ..
... ..
... ..
... ..

Output

... ..
... ..
... ..

Results

... ..
... ..
... ..

... .. Experiment - 10

... ..

... ..
... ..
... ..

to do:
to include <stdio.h>
to include <sys/types.h>
to include <unistd.h>
to include <fcntl.h>

static message &

long buffer

char text [100];

};

int main() {

key & key = 1234;

int msgid;

static message msg;

msgid = msgget(key, 0666) IPK_CREAT;

msg.type = 1;

strcpy(msg.text, "Hello from message
Queue");

msg.send(msgid, 0, size of (msg.text));

printf("message sent : 1.8 in", msg.text);
return 0;

}

Output:

message-lent : Hello from message queue

Result:

Thus, the concept of inter-process communication using a message queue was successfully illustrated using a C program.